# Fluid Instabilities

Conor Lewis
23904082
October 2026

**Abstract**

The onset of a flow instability in a shallow, electromagnetically driven electrolyte layer was investigated using surface particle tracking. Increasing forcing produced stronger transverse motion together with progressive distortion of the initially banded vorticity field. A continuous segmented orthogonal-distance regression gave a critical Reynolds number of $Re_{c,\uparrow} = 68.4^{+0.6}_{-0.9}$ for the increasing-current sequence and $Re_{c,\downarrow} = 59.2^{+0.9}_{-0.7}$ for the return sequence. Their separation, $\Delta Re_c = 9.2^{+0.9}_{-1.3}$, provides evidence of path dependence consistent with hysteresis. The increasing-current transition is comparable with previous measurements of Kolmogorov-like flow, although the absolute threshold remains sensitive to friction, geometry and the assumed viscosity.

## 1 Introduction

A flow instability occurs when a previously stable flow state becomes unable to suppress disturbances as a control parameter is varied. The resulting motion need not immediately become turbulent; an initially parallel flow may instead reorganise into a new steady structure. Kolmogorov flow provides a useful experimental example, in which spatially periodic forcing produces adjacent bands of oppositely directed motion [1].

This experiment investigated the first instability of a shallow electromagnetically driven layer using surface particle tracking. The flow was characterised using its total root-mean-square (RMS) velocity, transverse RMS velocity and spatial vorticity field. Measurements were made during both the increasing-current and return sequences. The principal aims were to estimate the critical Reynolds number for the first instability and to determine whether the transition depended on the direction from which it was approached.

## 2 Theory

An electric current passing through a conducting liquid in a magnetic field experiences a Lorentz force. The resulting body force per unit mass is

$$\mathbf{f}_B = \frac{\mathbf{J} \times \mathbf{B}}{\rho}, \tag{1}$$

where $\mathbf{J}$ is the current density, $\mathbf{B}$ is the magnetic field and $\rho$ is the fluid density. In the present experiment, neighbouring rows of magnets have alternating polarity. The Lorentz force therefore acts in opposite directions in adjacent regions of the fluid, producing an approximately parallel banded flow.

The balance between inertial motion and viscous damping is characterised by the Reynolds number,

$$Re = \frac{UL}{\nu}, \qquad U = \sqrt{\langle u_x^2 + u_y^2 \rangle}, \tag{2}$$

where $Re$ is the Reynolds number, $U$ is the RMS fluid speed, $L$ is the characteristic length scale of the forcing and $\nu$ is the kinematic viscosity. The quantities $u_x$ and $u_y$ are the measured fluid-velocity components in the $x$- and $y$-directions respectively, while the angle brackets denote averaging over the measured particle-velocity samples. Following Kelley and Ouellette [1], the $x$-direction is defined across the forcing bands and the $y$-direction along them. The characteristic length $L$ is taken to be the full spatial wavelength of the magnetic forcing.

At low Reynolds number, viscous damping is relatively strong and the approximately parallel banded flow remains stable. As $Re$ increases, inertial effects become more important relative to viscosity and the base flow can become unstable. The Reynolds number at which the first instability occurs is denoted by the critical Reynolds number, $Re_c$.

An increase in the total speed $U$ alone does not demonstrate that an instability has occurred, since the original banded flow could simply become faster as the forcing is increased. A more sensitive measure is therefore the RMS velocity across the forcing bands,

$$U_\perp = \sqrt{\langle u_x^2 \rangle}, \tag{3}$$

where $U_\perp$ is the cross-band, or transverse, RMS velocity. For an ideal parallel base flow, motion is entirely along the bands and $u_x = 0$, giving $U_\perp = 0$. In the real experiment a small non-zero value remains because of particle-tracking noise, finite boundaries and imperfect alignment of the coordinate axes. A sustained increase in $U_\perp$ therefore indicates the development of motion across the original flow bands and provides a useful measure of the onset of instability.

A complementary description of the changing flow structure is provided by the out-of-plane vorticity,

$$\omega_z = \frac{\partial u_y}{\partial x} - \frac{\partial u_x}{\partial y}, \tag{4}$$

where $\omega_z$ measures the local rotation of the two-dimensional velocity field. The base flow already contains alternating positive and negative vorticity because neighbouring fluid bands move in opposite directions. Instability is therefore identified not by the appearance of vorticity itself, but by distortion of the initially approximately straight and parallel vorticity bands. The growth of $U_\perp$ and distortion of the vorticity pattern thus provide two complementary signatures of the same transition.

The precise value of $Re_c$ is apparatus dependent. Bottom friction, finite confinement, fluid depth and deviations from ideal sinusoidal forcing can all alter the instability threshold [1, 3]. Kelley and Ouellette measured $Re_c = 61$ in a related electromagnetically driven flow, while Burgess et al. obtained approximately $Re_c = 70$ in a soap-film implementation [2]. These values therefore provide useful comparisons rather than universal predictions for the present apparatus.

If the transition occurs at different Reynolds numbers while the forcing is increased and decreased, the response is history-dependent. The separation between the measured transition points is

$$\Delta Re_c = Re_{c,\uparrow} - Re_{c,\downarrow}, \tag{5}$$

where $Re_{c,\uparrow}$ is the transition measured during the increasing-forcing sequence and $Re_{c,\downarrow}$ is that measured during the return sequence. A positive $\Delta Re_c$ indicates measurable path dependence and is consistent with hysteresis, in which the observed flow state depends not only on the instantaneous control parameter but also on the previous state of the system. Such memory and multistability have been observed in other nonlinear fluid flows [4]. However, finite relaxation times during a parameter sweep can also produce an apparent separation between the two thresholds. Repeated, progressively slower sweeps would therefore be required to distinguish intrinsic quasi-static hysteresis from dynamic lag.

# 3 Experimental Method

## 3.1 Apparatus

The apparatus was set up following the experiment of Kelley and Ouellette [1]. A tray measuring approximately 165 mm × 170 mm was filled to a depth of approximately 10 mm with a conducting liquid and positioned above rows of alternating-polarity magnets. Copper electrodes connected the fluid to a DC power supply through a series ammeter, allowing the electromagnetic forcing to be varied by changing the applied current.

The solution contained 10% copper sulphate by mass and 20% glycerol by preparation volume, using 60 mL glycerol and 240 mL deionised water. Polyamide tracer particles were distributed on the free surface, illuminated with blue light and recorded from above.

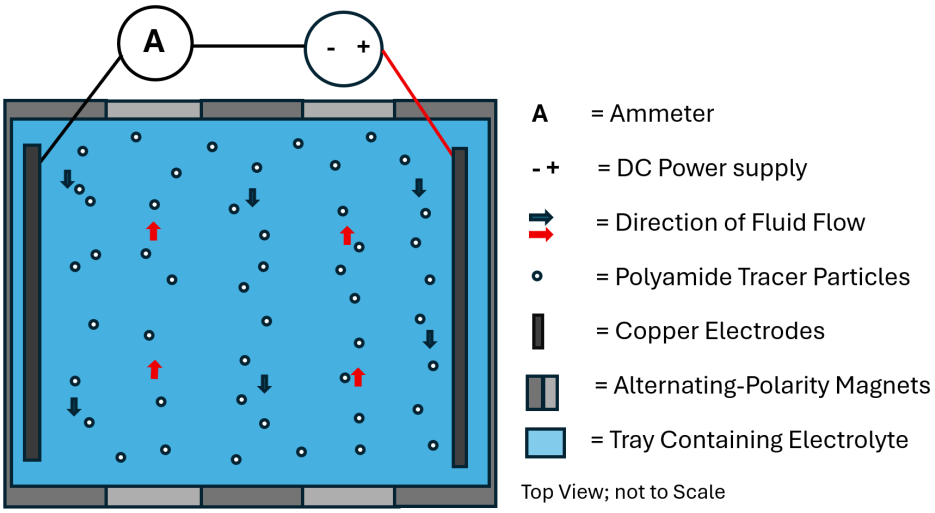



**Figure 1:** Top-view schematic of the experimental arrangement. Current passes between the copper electrodes through the electrolyte. Alternating-polarity magnets beneath the tray produce neighbouring bands of oppositely directed Lorentz forcing. Arrows show the idealised base-flow directions.

## 3.2 Acquisition and calibration

Videos were recorded at 1280 × 720 pixels and 30 Hz. Decoded frame timestamps were checked in two complete recordings and confirmed frame separations of approximately 33.3 ms (30 Hz).

Three approximately three-second intervals, near the beginning, middle and end of each recording, were analysed. This provided samples from different stages of each recording without allowing the longest or most densely tracked interval to dominate the calculated velocity statistics.

Spatial calibration was obtained from ruler inter-

vals measured within approximately 5 mm of the tracer height, giving a scale of 0.021 57 cm per pixel. Calibration measurements made near the upper and lower parts of the image differed by approximately 2%, indicating a small perspective effect. This variation was incorporated into the calibration uncertainty through the perspective term described in subsection 3.6, which also accounts for uncertainty in the measured pixel-to-ruler scale ratio.

The ammeter resolution was 0.05 mA.

### 3.3 Particle tracking and derived quantities

Bright tracer candidates were isolated after spatial background suppression and their positions refined using intensity-weighted centroids. Detections were linked between frames using constant-velocity prediction. Ambiguous matches were rejected and tracks ended following missed detections. Only tracks containing at least 15 observations within the selected image region were retained.

The coordinate system was rotated by 4.77°, determined from the orientation of the low-current flow bands, so that the $y$-direction followed the base-flow direction and the $x$-direction lay across the forcing bands. Any residual alignment error contributes to the small non-zero transverse velocity measured below the transition. Particle velocities were obtained from linear fits to position over 11 observations, with estimates evaluated every five observations.

RMS velocities were calculated separately for each of the three temporal windows and then averaged with equal weighting. Reynolds numbers were calculated from Equation 2 using

$$L = 0.050\,\mathrm{m}, \qquad \nu = 2.61 \times 10^{-6}\,\mathrm{m^2\,s^{-1}},$$

where $L$ is the full spatial wavelength of the repeating magnetic forcing pattern and the kinematic viscosity was taken from Kelley and Ouellette [1].

Vorticity was estimated by fitting local planar velocity fields within a 0.65 cm radius and evaluating the spatial derivatives in Equation 4. Maps from the three temporal windows were averaged with equal weighting. Locations without sufficient particle support were left blank.

### 3.4 Increasing-current and return sequences

Current was initially increased through the experimental range to a maximum of 80 mA. A return sequence was subsequently recorded with decreasing current to determine whether the flow retraced the same transition.

For both sequences, approximately 15 s was allowed for the flow to settle after each change in current before a 15 s recording was made. Only one complete increasing-current and return cycle was performed, so any observed path dependence cannot definitively distinguish quasi-static hysteresis from finite-time response effects.

### 3.5 Breakpoint and uncertainty analysis

Kelley and Ouellette represented their transverse-velocity onset using a constant noise floor below $Re_c$ followed by an approximate square-root increase above the transition [1]. This was intended as a crude near-onset description rather than a first-principles prediction of the bifurcation shape.

The present measurements showed a non-zero transverse baseline together with a gradual pre-transition trend. Applying the same square-root form would therefore require the low-$Re$ response to be approximately constant and could make the inferred onset sensitive to an assumption not well satisfied by these data. Instead, the transition was located using the continuous segmented model

$$U_\perp = a + bRe + c\max(0, Re - Re_b), \qquad (6)$$

where $Re_b$ is the fitted change point. The model is used only as an empirical estimator of the location at which the slope changes; it is not interpreted as the physical form of the bifurcation. The fitted $Re_b$ is consequently reported as the experimental estimate of $Re_c$.

Orthogonal-distance regression (ODR) was used so that the assigned uncertainties in both $Re$ and $U_\perp$ entered the fit. For each trial breakpoint, ODR determined the remaining three coefficients and the breakpoint giving the minimum weighted orthogonal residual sum was selected. The final fit used 1000 trial breakpoint positions.

Breakpoint uncertainty was estimated using 500 Monte Carlo realisations. In each realisation, the measured $Re$ and $U_\perp$ values were independently perturbed according to their assigned Gaussian uncertainties, resorted in $Re$, and the breakpoint search repeated using 240 trial values. The 16th and 84th percentiles of the resulting distribution were used as the central 68% Monte Carlo interval.

For the fit diagnostics, residual standard deviations were calculated separately below and above the fitted breakpoint. The shaded regions in the breakpoint figures show these empirical $\pm 1\sigma$ residual spreads. Residuals were used only to assess the local adequacy of the segmented fit and were not used to determine the breakpoint.

### 3.6 Measurement uncertainty

The standard error across the three sampled temporal windows was combined in quadrature with the spatial calibration uncertainty. Calibration contributions included an endpoint reading uncertainty of two pixels, a 0.5% standard uncertainty associated with the ruler-height mismatch and a uniform ±1% allowance for perspective-related variation in the pixel-to-ruler scale ratio.

Treating the 0.05 mA ammeter resolution as a rectangular distribution gives

$$u_I = \frac{0.05}{\sqrt{12}} \simeq 0.014\,\text{mA}.$$

The forcing wavelength and literature viscosity were treated as fixed. The plotted Reynolds-number uncertainties therefore represent propagated measurement uncertainty rather than a complete uncertainty in the absolute Reynolds-number scale. In particular, uncertainty in viscosity, finite-depth effects and other apparatus-specific effects are better treated as systematic limitations of the absolute value of $Re_c$.

# 4 Results

## 4.1 Reynolds-number response

Figure 2 shows the measured Reynolds number as the applied current was increased and subsequently decreased. Greater current generally produced greater flow speed and hence greater $Re$, but the return sequence did not exactly retrace the relationship between applied current and $Re$ measured during the increasing-current sequence.

For this reason, applied current was treated only as the experimental forcing control. The instability itself was analysed against the measured Reynolds number rather than assuming a fixed conversion between current and $Re$.

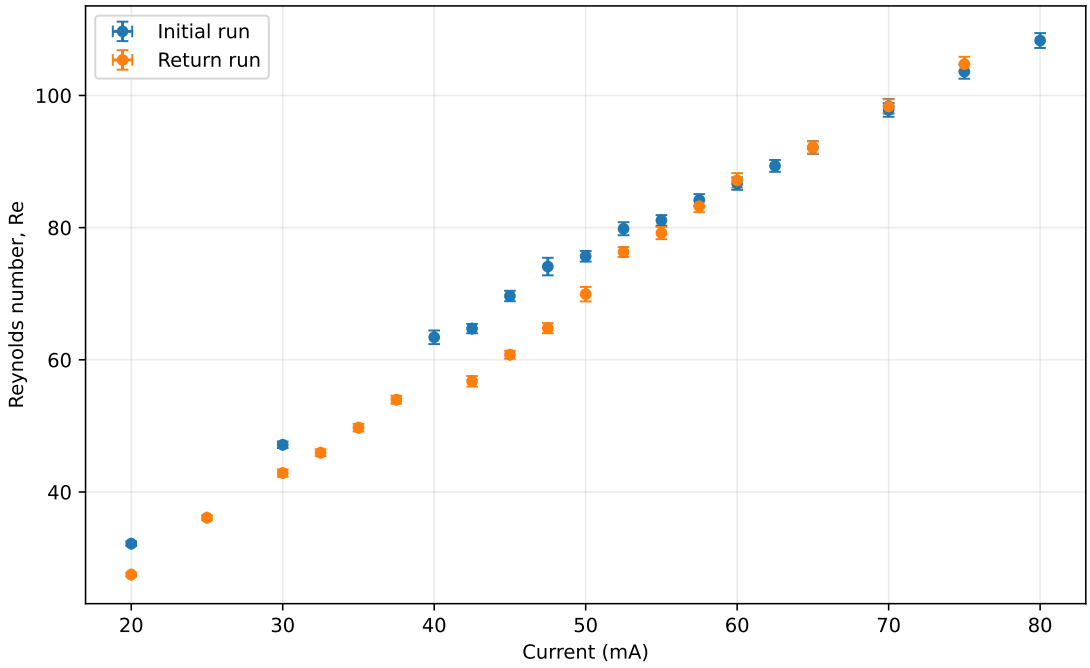



**Figure 2:** Measured Reynolds number as a function of applied current for the increasing-current and return sequences. Error bars show propagated measurement uncertainty.

## 4.2 Flow structure and transverse motion

At low Reynolds number, the vorticity field retained the approximately parallel alternating-band structure expected for the base flow. As $Re$ increased, these bands became progressively displaced and distorted.

Figure 3 was selected to show the flow below, close to and above the measured transition rather than at equally spaced currents. In particular, panels (b) and (c), at $Re = 63.4$ and 69.7, lie immediately on either side of the fitted increasing-current critical value. The structural change across these panels provides a qualitative check that the breakpoint inferred from $U_\perp$ corresponds to a genuine reorganisation of the flow.

The same transition is evident quantitatively in Figure 4. At low $Re$, $U_\perp$ remains close to a small residual baseline. It then rises more rapidly as significant motion develops across the original forcing bands.

The return data remain elevated to lower Reynolds number than the increasing-current data. Thus, over part of the transition region, flows at similar $Re$ can possess different transverse amplitudes depending on the direction from which that Reynolds number was reached.

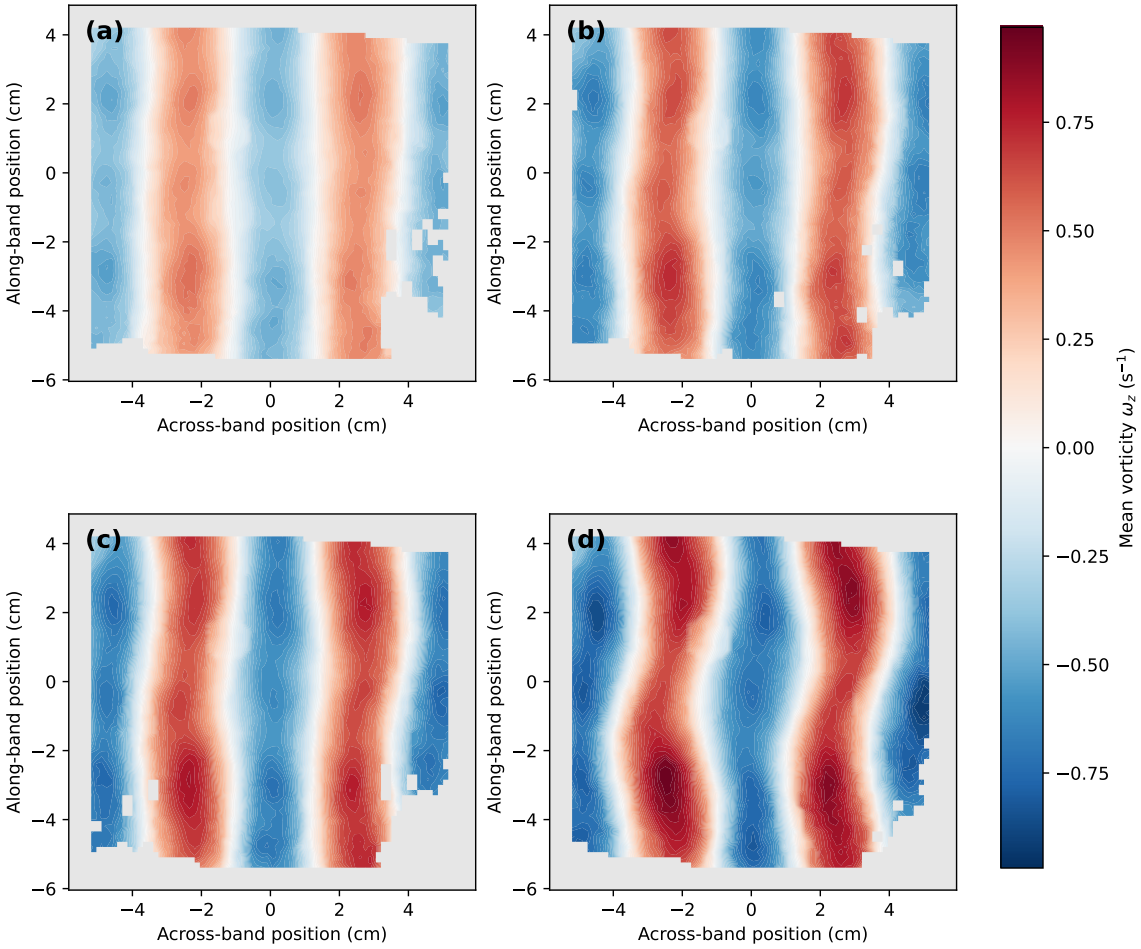



**Figure 3:** Mean vorticity fields during the increasing-current sequence at (a) $Re = 47.1$, (b) $Re = 63.4$, (c) $Re = 69.7$ and (d) $Re = 79.8$, corresponding to currents of 30, 40, 45 and 52.5 mA respectively. Red and blue denote opposite signs of $\omega_z$, while grey regions denote insufficient particle support. Progressive distortion of the alternating bands is visible as the flow passes through the measured instability.

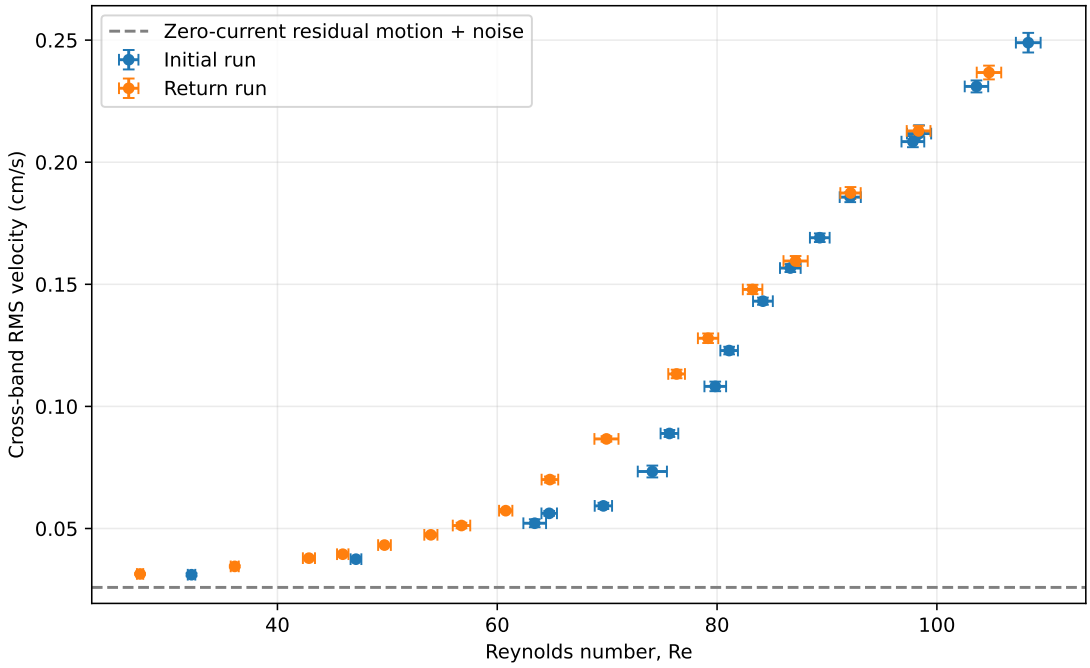



**Figure 4:** Cross-band RMS velocity $U_\perp$ against Reynolds number for the increasing-current and return sequences. The dashed horizontal line shows the residual transverse motion measured at zero applied current.

### 4.3 Critical Reynolds numbers and path dependence

For the increasing-current sequence, the segmented ODR fit gave

$$\boxed{Re_{c,\uparrow} = 68.4^{+0.6}_{-0.9}}. \tag{7}$$

The central 68% Monte Carlo interval was 67.5–69.0.

The breakpoint uncertainties reported here describe the statistical uncertainty under the assigned measurement uncertainties. They do not include systematic uncertainty in the absolute Reynolds-number scale arising, for example, from the assumed viscosity or characteristic forcing length.

The fit and residuals are shown in Figure 5. Their behaviour is interpreted in the Discussion.

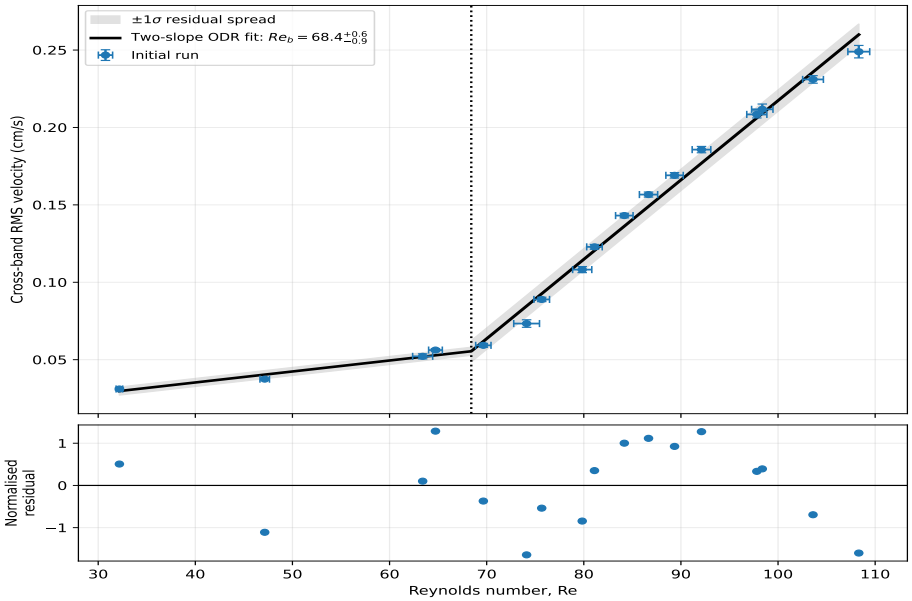



**Figure 5:** Segmented ODR fit to the increasing-current sequence. Blue points show the measurements, the solid black line is the empirical two-slope fit and the dotted line marks the fitted breakpoint. The shaded region shows the empirical $\pm 1\sigma$ residual spread on each branch; normalised residuals are shown below.

Applying the same procedure to the return sequence gave

$$\boxed{Re_{c,\downarrow} = 59.2^{+0.9}_{-0.7}}. \tag{8}$$

The central 68% Monte Carlo interval was 58.5–60.1. The fit and residuals are shown in Figure 6.

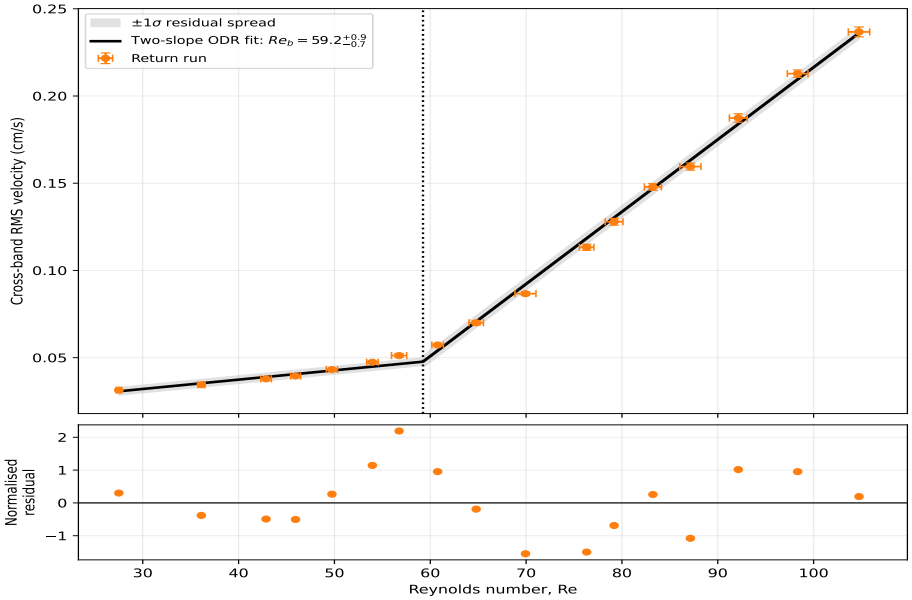



**Figure 6:** Equivalent segmented ODR analysis of the return sequence. Orange points show the measurements, while the empirical two-slope fit and fitted breakpoint are shown in black. The transition occurs at a lower Reynolds number than during the increasing-current sequence.

The fitted transition separation is

$$\Delta Re_c = 68.4 - 59.2 = 9.2. \tag{9}$$

Treating the statistical breakpoint uncertainties of the two sequences as independent and propagating the asymmetric limits in quadrature gives

$$\boxed{\Delta Re_c = 9.2^{+0.9}_{-1.3}}. \tag{10}$$

# 5 Discussion

## 5.1 Critical Reynolds number and comparison with literature

The primary aim of the experiment was to identify the onset of the first instability in the electromagnetically driven flow. This was achieved using two complementary indicators: the growth of the cross-band RMS velocity and the accompanying distortion of the initially banded vorticity field. The increasing-current sequence gave

$$Re_{c,\uparrow} = 68.4^{+0.6}_{-0.9},$$

placing the transition in the same range as previous Kolmogorov-flow experiments. Kelley and Ouellette measured $Re_c = 61$ in a closely related undergraduate apparatus, while Burgess et al. reported a value of approximately $Re_c = 70$ in a soap-film experiment [1, 2].

Exact agreement between these values is not expected. The critical Reynolds number depends on the physical realisation of the flow, including bottom friction, fluid depth, confinement and the spatial forcing profile. Tithof et al. showed that realistic boundary conditions and forcing must be included to reproduce quasi-two-dimensional Kolmogorov-flow experiments quantitatively [3]. The agreement in scale is therefore more significant than exact numerical agreement and supports the identification of the measured breakpoint with the expected first instability.

The agreement between the transverse-velocity and vorticity measurements provides a complementary structural check on this interpretation. Figure 4 shows the rapid growth of transverse motion, while Figure 3 shows that the alternating vorticity bands lose their approximately parallel form over the same Reynolds-number range. The inferred breakpoint therefore corresponds to a visible reorganisation of the flow rather than merely an increase in the speed of the original base state.

## 5.2 Shape of the transverse-velocity response

The form of $U_\perp(Re)$ also explains the choice of breakpoint model. Kelley and Ouellette represented the onset using an approximately constant pre-transition noise floor followed by square-root growth [1]. In the present experiment, however, $U_\perp$ already increases gradually below the main transition. A model requiring a strictly constant pre-transition value would therefore not describe the measured baseline well.

The continuous segmented fit instead allows one gradient below the transition and another above it. It is not intended as a theoretical model of the bifurcation, but as an empirical method of locating the clear change in gradient. The residuals support this use of the model. Close to the fitted breakpoints, both sequences show small residuals without strong systematic structure, so the local change in slope is well resolved (Figure 5 and Figure 6).

The two sequences differ more noticeably at larger Reynolds number. The return sequence remains close to a linear post-transition trend over most of the measured range, whereas the increasing-current sequence develops clear upward curvature. Correspondingly, the increasing-current residuals become increasingly structured away from the transition. At greater forcing the flow may undergo further changes or secondary instabilities, as observed in more detailed studies of quasi-two-dimensional Kolmogorov flow [3].

The different shapes of the increasing-current and return responses also show that the two sequences are not simply identical curves shifted horizontally. The state reached at a given Reynolds number depends to some extent on the previous evolution of the flow, consistent with the path dependence found from the fitted transition values.

## 5.3 Path dependence and experimental improvements

The return sequence gave

$$Re_{c,\downarrow} = 59.2^{+0.9}_{-0.7},$$

approximately nine Reynolds-number units below the increasing-current transition. The difference,

$$\Delta Re_c = 9.2^{+0.9}_{-1.3},$$

is substantially larger than the propagated breakpoint-fit uncertainty. Once the distorted flow state had formed, it therefore remained observable at Reynolds numbers below those required to create it during the increasing sequence. This is clear evidence of path dependence consistent with hysteresis. Similar memory and multistability have been observed in other nonlinear flows [4].

The present experiment is not sufficient, however, to establish that the full measured separation represents intrinsic quasi-static hysteresis. Only one complete increasing-current and return cycle was recorded,

The separation is much larger than the breakpoint-fit uncertainties. The post-transition state therefore persisted to a lower measured Reynolds number during the return sequence than was required to establish it during the increasing-current sequence. This provides clear experimental evidence of path dependence and is consistent with hysteretic behaviour.

and approximately 15 s was allowed for the flow to settle after each change in current. If this is comparable with the relaxation time of the flow, finite response time could shift the apparent transition during the sweep.

A more complete investigation would therefore repeat the complete increasing-current and return sequence several times and compare different sweep rates and settling times. If the separation between the two thresholds remained as the sweep was made progressively slower, the evidence for genuine hysteresis would be much stronger. Additional measurements concentrated near the two transition regions would also improve localisation of the breakpoints without requiring substantially more data in the fully developed regime.

The current–Reynolds-number relation in Figure 2 also differed slightly between the two sequences. Because the instability analysis uses measured $Re$ rather than current, differences in flow speed at a given current are already largely accounted for. Nevertheless, this variation suggests that the experimental conditions may have evolved during the measurement sequence. In particular, the viscosity was taken from the literature and treated as constant. Measuring the solution temperature and viscosity directly would reduce uncertainty in the absolute Reynolds-number scale and help determine whether time-dependent viscosity changes contributed to the apparent separation between the increasing-current and return thresholds.

### 5.4 Broader relevance

The method used here, varying a dimensionless control parameter and identifying the point at which a flow changes state, is widely applicable in fluid mechanics. A critical dimensionless number allows transitions to be compared between systems with different fluids, sizes and operating conditions.

Rayleigh–Bénard convection provides a classical example. A fluid heated from below remains in a conductive state until the Rayleigh number exceeds a critical value, after which organised convection develops [5]. As in the present experiment, the transition can be identified by monitoring a quantity that is small in the original state and grows after instability. Such threshold behaviour is relevant to laboratory systems as well as geophysical and astrophysical heat transport.

Pipe flow demonstrates why experimentally measured transition thresholds must be interpreted with reference to the physical system. Laminar Hagen–Poiseuille flow is linearly stable, yet sufficiently large disturbances can trigger turbulence, so the observed transition depends on both Reynolds number and disturbance amplitude [6]. Similarly, the critical value measured here characterises the particular experimental realisation rather than a universal constant independent of boundaries and forcing.

Finally, the observed dependence on sweep direction illustrates why hysteresis is important in practical flows. During dynamic stall of an aerofoil, flow separation and subsequent reattachment occur under different conditions as the angle of attack is increased and decreased [7]. History dependence of this type is important in systems such as wind turbines and helicopter rotors because the instantaneous control parameter alone may not determine the current aerodynamic state.

The present experiment therefore demonstrates two broadly useful ideas: locating the onset of instability through a dimensionless control parameter, and testing whether the resulting flow state depends only on that parameter or also on the path by which it was reached.

## 6 Conclusion

Surface particle tracking resolved the first instability of an electromagnetically driven Kolmogorov-like flow using both spatial and statistical measures. As the Reynolds number increased, the initially banded vorticity structure became distorted while the cross-band RMS velocity increased rapidly.

The increasing-current transition was measured as

$$Re_{c,\uparrow} = 68.4^{+0.6}_{-0.9},$$

while the return transition occurred at

$$Re_{c,\downarrow} = 59.2^{+0.9}_{-0.7}.$$

The resulting separation,

$$\Delta Re_c = 9.2^{+0.9}_{-1.3},$$

is substantially larger than the breakpoint-fit uncertainty and demonstrates path dependence consistent with hysteresis.

The increasing-current threshold agrees in scale with previous Kolmogorov-flow measurements, while differences in friction, geometry and viscosity prevent the absolute value from being treated as universal. The principal limitation of the hysteresis interpretation is the single complete increasing-current and return cycle and the finite settling time. Repeated sweeps at different rates would therefore provide the most useful extension of the experiment.

## References


[1] D. H. Kelley and N. T. Ouellette, “Using particle tracking to measure flow instabilities in an undergraduate laboratory experiment,” *American Journal of Physics* **79**, 267–273 (2011). doi:10.1119/1.3536647.

[2] J. M. Burgess, C. Bizon, W. D. McCormick, J. B. Swift and H. L. Swinney, “Instability of the Kolmogorov flow in a soap film,” *Physical Review E* **60**, 715–721 (1999). doi:10.1103/PhysRevE.60.715.

[3] J. Tithof, B. Suri, R. K. Pallantla, R. O. Grigoriev and M. F. Schatz, “Bifurcations in

a quasi-two-dimensional Kolmogorov-like flow," *Journal of Fluid Mechanics* **828**, 837–866 (2017). doi:10.1017/jfm.2017.553.

[4] F. Ravelet, L. Marié, A. Chiffaudel and F. Daviaud, "Multistability and memory effect in a highly turbulent flow: experimental evidence for a global bifurcation," *Physical Review Letters* **93**, 164501 (2004). doi:10.1103/PhysRevLett.93.164501.

[5] E. Bodenschatz, W. Pesch and G. Ahlers, "Recent developments in Rayleigh–Bénard convection," *Annual Review of Fluid Mechanics* **32**, 709–778 (2000). doi:10.1146/annurev.fluid.32.1.709.

[6] V. Mukund and B. Hof, "The critical point of the transition to turbulence in pipe flow," *Journal of Fluid Mechanics* **839**, 76–94 (2018). doi:10.1017/jfm.2017.923.

[7] S. Rezapour and K. Mulleners, "Dynamic stall reattachment revisited," *Journal of Fluid Mechanics* **1029**, A52 (2026). doi:10.1017/jfm.2026.11228.

# A Computational Appendix

The following pages contain the computational analysis used for particle tracking, uncertainty propagation, Reynolds-number calculation, vorticity reconstruction and breakpoint fitting. Large parts of the analysis workflow were adapted from MATLAB code provided by Kelley and Ouellette [1], with further processing and statistical analysis implemented in Python for the present experiment.

# File saving and initial treament

In [1]:
```
# Filename: current in mA.
# Original files are not renamed.

runs = {
    "Initial run": {
        "20 (45).mp4": 20.0,
        "30(47).mp4": 30.0,
        "40(48).mp4": 40.0,
        "45(49).mp4": 45.0,
        "47.5(50).mp4": 42.5,
        "42.5(51).mp4": 47.5,
        "50(52).mp4": 50.0,
        "52.5(54).mp4": 52.5,
        "55(56).mp4": 55.0,
        "57.5(58).mp4": 57.5,
        "60m(59).mp4": 62.5,
        "60m(0).mp4": 60.0,
        "60m(2).mp4": 65.0,
        "65(3).mp4": 70.0,
        "75(4).mp4": 75.0,
        "70(5).mp4": 70.0,
        "80(6).mp4": 80.0,
    },

    "Return run": {
        "70(7).mp4": 70.0,
        "75(8).mp4": 75.0,
        "65(10).mp4": 65.0,
        "60(11).mp4": 60.0,
        "57.5(11).mp4": 57.5,
        "55(13).mp4": 55.0,
        "52.5(14).mp4": 52.5,
        "50(16).mp4": 50.0,
        "47.5(17).mp4": 47.5,
        "45(18).mp4": 45.0,
        "42.5(19).mp4": 42.5,
        "37.5(20).mp4": 37.5,
        "35(21).mp4": 35.0,
        "32.5(22).mp4": 32.5,
        "30(23).mp4": 30.0,
        "25(24).mp4": 25.0,
        "20(25).mp4": 20.0,
    },
}

zero_reference = "0(25) - Copy.mp4"
uncertain_zero = "20.mp4"

# Combined list compatible with the earlier batch-tracking cell.
recordings = [
    (filename, current)
    for files in runs.values()
    for filename, current in files.items()
] + [(zero_reference, 0.0)]
```

# Rotation of Cells

In [2]:
```
from pathlib import Path
import numpy as np
import pandas as pd

DATA_FOLDER = Path(
    r"C:\Users\lewis\OneDrive\Documents\Lab reports"
    r"\New folder\particle_tracking_results"
)

ANGLE_DEG = 4.7667
theta = np.deg2rad(ANGLE_DEG)

# Origin within the analysed region
X_ORIGIN = 670.0
Y_ORIGIN = 370.0

positions_by_file = {}

for filename, current in recordings:
    csv_path = DATA_FOLDER / (
        f"{Path(filename).stem}_sampled_positions.csv"
    )

    df = pd.read_csv(csv_path)
    df = df.sort_values(
        ["window", "track", "time_s"]
    ).reset_index(drop=True)

    dx = df["x_px"] - X_ORIGIN
    dy = df["y_px"] - Y_ORIGIN

    # Across-band coordinate and upward along-band coordinate.
    df["cross_px"] = np.cos(theta) * dx - np.sin(theta) * dy
    df["along_px"] = -(
        np.sin(theta) * dx + np.cos(theta) * dy
    )

    # Preserve original x_px and y_px.
    positions_by_file[filename] = df

print(
    f"Loaded and rotated {len(positions_by_file)} recordings "
    f"by {ANGLE_DEG:.4f}°."
)
```

```
Loaded and rotated 35 recordings by 4.7667°.
```

# Recording of Velocities by Particle Tracking

In [3]:
```
CM_PER_PIXEL = 0.02157156
FIT_LENGTH = 11
FIT_STRIDE = 5
```

```
OUTPUT_FOLDER = DATA_FOLDER / "rotated_results"
OUTPUT_FOLDER.mkdir(parents=True, exist_ok=True)

# Keep the final current assignments from Cell 1.
recordings = [
    (filename, current)
    for files in runs.values()
    for filename, current in files.items()
] + [(zero_reference, 0.0)]

run_by_file = {
    filename: run
    for run, files in runs.items()
    for filename in files
}
run_by_file[zero_reference] = "Zero reference"

summary_rows = []
velocities_by_file = {}

for filename, current in recordings:
    positions = positions_by_file[filename]
    velocity_rows = []

    for (window, track_id), track in positions.groupby(
        ["window", "track"]
    ):
        track = track.sort_values("time_s")

        times = track["time_s"].to_numpy()
        xy = track[["cross_px", "along_px"]].to_numpy()

        if np.any(np.diff(times) <= 0):
            raise ValueError(f"Invalid track timestamps: {filename}")

        for start in range(
            0, len(track) - FIT_LENGTH + 1, FIT_STRIDE
        ):
            stop = start + FIT_LENGTH

            t = times[start:stop]
            coordinates = xy[start:stop]

            dt = t - t.mean()

            # Local linear fit: pixels/second, then cm/second.
            velocity = (
                dt[:, None]
                * (coordinates - coordinates.mean(axis=0))
            ).sum(axis=0) / np.sum(dt**2)

            u_cross, u_along = velocity * CM_PER_PIXEL
            cross_cm, along_cm = (
                coordinates.mean(axis=0) * CM_PER_PIXEL
            )

            velocity_rows.append({
                "window": window,
                "track": track_id,
```

```
                "time_s": t.mean(),
                "cross_cm": cross_cm,
                "along_cm": along_cm,
                "u_cross_cm_s": u_cross,
                "u_along_cm_s": u_along,
            })

    if not velocity_rows:
        raise ValueError(f"No usable velocities: {filename}")

    velocities = pd.DataFrame(velocity_rows)

    velocities["speed_cm_s"] = np.hypot(
        velocities["u_cross_cm_s"],
        velocities["u_along_cm_s"]
    )

    # RMS within each sampled window.
    window_rms = velocities.groupby("window")["speed_cm_s"].apply(
        lambda v: np.sqrt(np.mean(v**2))
    )

    window_cross_rms = velocities.groupby("window")[
        "u_cross_cm_s"
    ].apply(lambda v: np.sqrt(np.mean(v**2)))

    # Retain results for subsequent plots.
    velocities_by_file[filename] = velocities

    stem = Path(filename).stem

    positions.to_csv(
        OUTPUT_FOLDER / f"{stem}_rotated_positions.csv",
        index=False
    )

    velocities.to_csv(
        OUTPUT_FOLDER / f"{stem}_rotated_velocities.csv",
        index=False
    )

    summary_rows.append({
        "run": run_by_file[filename],
        "recording": filename,
        "current_mA": current,
        "mean_window_rms_cm_s": window_rms.mean(),
        "minimum_window_rms_cm_s": window_rms.min(),
        "maximum_window_rms_cm_s": window_rms.max(),
        "cross_rms_cm_s": window_cross_rms.mean(),
        "windows_used": len(window_rms),
    })

    print(
        f"{filename}: "
        f"total RMS = {window_rms.mean():.3f} cm/s; "
        f"cross-band RMS = {window_cross_rms.mean():.3f} cm/s"
    )

summary = pd.DataFrame(summary_rows)
```

```
summary.to_csv(
    OUTPUT_FOLDER / "rotated_speed_summary.csv",
    index=False
)

display(summary.sort_values(["run", "current_mA"]))
```

```
20 (45).mp4: total RMS = 0.168 cm/s; cross-band RMS = 0.031 cm/s
30(47).mp4: total RMS = 0.246 cm/s; cross-band RMS = 0.037 cm/s
40(48).mp4: total RMS = 0.331 cm/s; cross-band RMS = 0.052 cm/s
45(49).mp4: total RMS = 0.364 cm/s; cross-band RMS = 0.059 cm/s
47.5(50).mp4: total RMS = 0.338 cm/s; cross-band RMS = 0.056 cm/s
42.5(51).mp4: total RMS = 0.387 cm/s; cross-band RMS = 0.073 cm/s
50(52).mp4: total RMS = 0.395 cm/s; cross-band RMS = 0.089 cm/s
52.5(54).mp4: total RMS = 0.417 cm/s; cross-band RMS = 0.108 cm/s
55(56).mp4: total RMS = 0.423 cm/s; cross-band RMS = 0.123 cm/s
57.5(58).mp4: total RMS = 0.439 cm/s; cross-band RMS = 0.143 cm/s
60m(59).mp4: total RMS = 0.466 cm/s; cross-band RMS = 0.169 cm/s
60m(0).mp4: total RMS = 0.452 cm/s; cross-band RMS = 0.157 cm/s
60m(2).mp4: total RMS = 0.481 cm/s; cross-band RMS = 0.186 cm/s
65(3).mp4: total RMS = 0.514 cm/s; cross-band RMS = 0.212 cm/s
75(4).mp4: total RMS = 0.541 cm/s; cross-band RMS = 0.231 cm/s
70(5).mp4: total RMS = 0.511 cm/s; cross-band RMS = 0.208 cm/s
80(6).mp4: total RMS = 0.565 cm/s; cross-band RMS = 0.249 cm/s
70(7).mp4: total RMS = 0.547 cm/s; cross-band RMS = 0.237 cm/s
75(8).mp4: total RMS = 0.513 cm/s; cross-band RMS = 0.213 cm/s
65(10).mp4: total RMS = 0.481 cm/s; cross-band RMS = 0.187 cm/s
60(11).mp4: total RMS = 0.455 cm/s; cross-band RMS = 0.160 cm/s
57.5(11).mp4: total RMS = 0.434 cm/s; cross-band RMS = 0.148 cm/s
55(13).mp4: total RMS = 0.413 cm/s; cross-band RMS = 0.128 cm/s
52.5(14).mp4: total RMS = 0.398 cm/s; cross-band RMS = 0.113 cm/s
50(16).mp4: total RMS = 0.365 cm/s; cross-band RMS = 0.087 cm/s
47.5(17).mp4: total RMS = 0.338 cm/s; cross-band RMS = 0.070 cm/s
45(18).mp4: total RMS = 0.317 cm/s; cross-band RMS = 0.057 cm/s
42.5(19).mp4: total RMS = 0.296 cm/s; cross-band RMS = 0.051 cm/s
37.5(20).mp4: total RMS = 0.282 cm/s; cross-band RMS = 0.047 cm/s
35(21).mp4: total RMS = 0.260 cm/s; cross-band RMS = 0.043 cm/s
32.5(22).mp4: total RMS = 0.240 cm/s; cross-band RMS = 0.039 cm/s
30(23).mp4: total RMS = 0.224 cm/s; cross-band RMS = 0.038 cm/s
25(24).mp4: total RMS = 0.189 cm/s; cross-band RMS = 0.034 cm/s
20(25).mp4: total RMS = 0.144 cm/s; cross-band RMS = 0.031 cm/s
0(25) - Copy.mp4: total RMS = 0.039 cm/s; cross-band RMS = 0.026 cm/s
```

| | run | recording | current_mA | mean_window_rms_cm_s | minimum_window_rms_c |
|---|---|---|---|---|---|
| **0** | Initial run | 20 (45).mp4 | 20.0 | 0.167998 | 0.166 |
| **1** | Initial run | 30(47).mp4 | 30.0 | 0.246101 | 0.244 |
| **2** | Initial run | 40(48).mp4 | 40.0 | 0.330993 | 0.326 |
| **4** | Initial run | 47.5(50).mp4 | 42.5 | 0.337876 | 0.335 |
| **3** | Initial run | 45(49).mp4 | 45.0 | 0.363606 | 0.359 |
| **5** | Initial run | 42.5(51).mp4 | 47.5 | 0.386861 | 0.379 |
| **6** | Initial run | 50(52).mp4 | 50.0 | 0.394979 | 0.391 |
| **7** | Initial run | 52.5(54).mp4 | 52.5 | 0.416747 | 0.412 |
| **8** | Initial run | 55(56).mp4 | 55.0 | 0.423332 | 0.422 |
| **9** | Initial run | 57.5(58).mp4 | 57.5 | 0.439379 | 0.435 |
| **11** | Initial run | 60m(0).mp4 | 60.0 | 0.452392 | 0.448 |
| **10** | Initial run | 60m(59).mp4 | 62.5 | 0.466378 | 0.465 |
| **12** | Initial run | 60m(2).mp4 | 65.0 | 0.480855 | 0.478 |
| **13** | Initial run | 65(3).mp4 | 70.0 | 0.513534 | 0.508 |
| **15** | Initial run | 70(5).mp4 | 70.0 | 0.510597 | 0.506 |
| **14** | Initial run | 75(4).mp4 | 75.0 | 0.540802 | 0.538 |
| **16** | Initial run | 80(6).mp4 | 80.0 | 0.565400 | 0.561 |
| **33** | Return run | 20(25).mp4 | 20.0 | 0.143559 | 0.142 |
| **32** | Return run | 25(24).mp4 | 25.0 | 0.188516 | 0.187 |
| **31** | Return run | 30(23).mp4 | 30.0 | 0.223761 | 0.220 |
| **30** | Return run | 32.5(22).mp4 | 32.5 | 0.239819 | 0.237 |

| | run | recording | current_mA | mean_window_rms_cm_s | minimum_window_rms_c |
|---|---|---|---|---|---|
| **29** | Return run | 35(21).mp4 | 35.0 | 0.259641 | 0.257 |
| **28** | Return run | 37.5(20).mp4 | 37.5 | 0.281671 | 0.279 |
| **27** | Return run | 42.5(19).mp4 | 42.5 | 0.296221 | 0.290 |
| **26** | Return run | 45(18).mp4 | 45.0 | 0.317225 | 0.316 |
| **25** | Return run | 47.5(17).mp4 | 47.5 | 0.338209 | 0.334 |
| **24** | Return run | 50(16).mp4 | 50.0 | 0.365062 | 0.358 |
| **23** | Return run | 52.5(14).mp4 | 52.5 | 0.398384 | 0.397 |
| **22** | Return run | 55(13).mp4 | 55.0 | 0.413335 | 0.409 |
| **21** | Return run | 57.5(11).mp4 | 57.5 | 0.434468 | 0.431 |
| **20** | Return run | 60(11).mp4 | 60.0 | 0.454899 | 0.447 |
| **19** | Return run | 65(10).mp4 | 65.0 | 0.480982 | 0.479 |
| **17** | Return run | 70(7).mp4 | 70.0 | 0.546743 | 0.543 |
| **18** | Return run | 75(8).mp4 | 75.0 | 0.513313 | 0.509 |
| **34** | Zero reference | 0(25) - Copy.mp4 | 0.0 | 0.039028 | 0.036 |

# Uncertainty Treatment and Calculations

In [6]:
```
# Preparation volumes and standard uncertainties, in mL.
Vg, Vw = 60.0, 240.0
dVg, dVw = 0.1, 1.0

fraction = Vg / (Vg + Vw)
d_fraction = np.sqrt(
    (Vw * dVg)**2 + (Vg * dVw)**2
) / (Vg + Vw)**2

print(
    f"Glycerol: {100*fraction:.2f}% "
    f"± {100*d_fraction:.3f} percentage points"
```

```
)

# Current uncertainty from the 0.05 mA display resolution.
d_current = 0.05 / np.sqrt(12)

# Assumed relative standard uncertainties in the calibration.
height_error = 0.005
reading_error = np.sqrt(2) * 2 / 463
ruler_error = 0.01 / 10
perspective_error = 0.01 / np.sqrt(3)  # Uniform allowance of ±1% from camera ti

scale_error = np.sqrt(
    height_error**2
    + reading_error**2
    + ruler_error**2
    + perspective_error**2
)

L = 0.050       # m
nu = 2.61e-6    # m²/s

rows = []

for filename, data in velocities_by_file.items():
    info = summary.loc[
        summary["recording"] == filename
    ].iloc[0]

    # RMS velocities for each sampled window.
    window_values = []

    for window, group in data.groupby("window"):
        speed = np.sqrt(np.mean(group["speed_cm_s"]**2))
        cross = np.sqrt(np.mean(group["u_cross_cm_s"]**2))
        window_values.append([speed, cross])

    window_values = np.array(window_values)
    n = len(window_values)

    speed, cross = window_values.mean(axis=0)

    # Standard errors assume independent windows and steady flow.
    if n > 1:
        speed_sd, cross_sd = window_values.std(axis=0, ddof=1)
        speed_sem = speed_sd / np.sqrt(n)
        cross_sem = cross_sd / np.sqrt(n)
    else:
        speed_sd = cross_sd = np.nan
        speed_sem = cross_sem = np.nan

    # Combine window scatter and calibration uncertainty.
    d_speed = np.hypot(speed_sem, speed * scale_error)
    d_cross = np.hypot(cross_sem, cross * scale_error)

    # Convert cm/s to m/s.
    Re = speed * 0.01 * L / nu
    d_Re = d_speed * 0.01 * L / nu

    rows.append({
        "run": info["run"],
```

```
        "recording": filename,
        "current_mA": info["current_mA"],
        "current_resolution_u_mA": d_current,
        "speed_cm_s": speed,
        "speed_window_SD": speed_sd,
        "speed_partial_u": d_speed,
        "cross_cm_s": cross,
        "cross_window_SD": cross_sd,
        "cross_partial_u": d_cross,
        "Re": Re,
        "Re_partial_u": d_Re
    })

uncertainties = pd.DataFrame(rows)

display(uncertainties.round(4))

uncertainties.to_csv(
    OUTPUT_FOLDER / "uncertainty_budget.csv",
    index=False
)

print(f"Relative calibration uncertainty: {100*scale_error:.2f}%")
print(f"Current resolution contribution: {d_current:.4f} mA")

# L and nu are held fixed
```

```
Glycerol: 20.00% ± 0.072 percentage points
```

| | run | recording | current_mA | current_resolution_u_mA | speed_cm_s | speed_winc |
|---|---|---|---|---|---|---|
| **0** | Initial run | 20 (45).mp4 | 20.0 | 0.0144 | 0.1680 | |
| **1** | Initial run | 30(47).mp4 | 30.0 | 0.0144 | 0.2461 | |
| **2** | Initial run | 40(48).mp4 | 40.0 | 0.0144 | 0.3310 | |
| **3** | Initial run | 45(49).mp4 | 45.0 | 0.0144 | 0.3636 | |
| **4** | Initial run | 47.5(50).mp4 | 42.5 | 0.0144 | 0.3379 | |
| **5** | Initial run | 42.5(51).mp4 | 47.5 | 0.0144 | 0.3869 | |
| **6** | Initial run | 50(52).mp4 | 50.0 | 0.0144 | 0.3950 | |
| **7** | Initial run | 52.5(54).mp4 | 52.5 | 0.0144 | 0.4167 | |
| **8** | Initial run | 55(56).mp4 | 55.0 | 0.0144 | 0.4233 | |
| **9** | Initial run | 57.5(58).mp4 | 57.5 | 0.0144 | 0.4394 | |
| **10** | Initial run | 60m(59).mp4 | 62.5 | 0.0144 | 0.4664 | |
| **11** | Initial run | 60m(0).mp4 | 60.0 | 0.0144 | 0.4524 | |
| **12** | Initial run | 60m(2).mp4 | 65.0 | 0.0144 | 0.4809 | |
| **13** | Initial run | 65(3).mp4 | 70.0 | 0.0144 | 0.5135 | |
| **14** | Initial run | 75(4).mp4 | 75.0 | 0.0144 | 0.5408 | |
| **15** | Initial run | 70(5).mp4 | 70.0 | 0.0144 | 0.5106 | |
| **16** | Initial run | 80(6).mp4 | 80.0 | 0.0144 | 0.5654 | |
| **17** | Return run | 70(7).mp4 | 70.0 | 0.0144 | 0.5467 | |
| **18** | Return run | 75(8).mp4 | 75.0 | 0.0144 | 0.5133 | |
| **19** | Return run | 65(10).mp4 | 65.0 | 0.0144 | 0.4810 | |
| **20** | Return run | 60(11).mp4 | 60.0 | 0.0144 | 0.4549 | |

| | run | recording | current_mA | current_resolution_u_mA | speed_cm_s | speed_winc |
|---|---|---|---|---|---|---|
| **21** | Return run | 57.5(11).mp4 | 57.5 | 0.0144 | 0.4345 | |
| **22** | Return run | 55(13).mp4 | 55.0 | 0.0144 | 0.4133 | |
| **23** | Return run | 52.5(14).mp4 | 52.5 | 0.0144 | 0.3984 | |
| **24** | Return run | 50(16).mp4 | 50.0 | 0.0144 | 0.3651 | |
| **25** | Return run | 47.5(17).mp4 | 47.5 | 0.0144 | 0.3382 | |
| **26** | Return run | 45(18).mp4 | 45.0 | 0.0144 | 0.3172 | |
| **27** | Return run | 42.5(19).mp4 | 42.5 | 0.0144 | 0.2962 | |
| **28** | Return run | 37.5(20).mp4 | 37.5 | 0.0144 | 0.2817 | |
| **29** | Return run | 35(21).mp4 | 35.0 | 0.0144 | 0.2596 | |
| **30** | Return run | 32.5(22).mp4 | 32.5 | 0.0144 | 0.2398 | |
| **31** | Return run | 30(23).mp4 | 30.0 | 0.0144 | 0.2238 | |
| **32** | Return run | 25(24).mp4 | 25.0 | 0.0144 | 0.1885 | |
| **33** | Return run | 20(25).mp4 | 20.0 | 0.0144 | 0.1436 | |
| **34** | Zero reference | 0(25) - Copy.mp4 | 0.0 | 0.0144 | 0.0390 | |

```
Relative calibration uncertainty: 0.98%
Current resolution contribution: 0.0144 mA
```

# Current vs Reynolds Number

In [8]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import odr

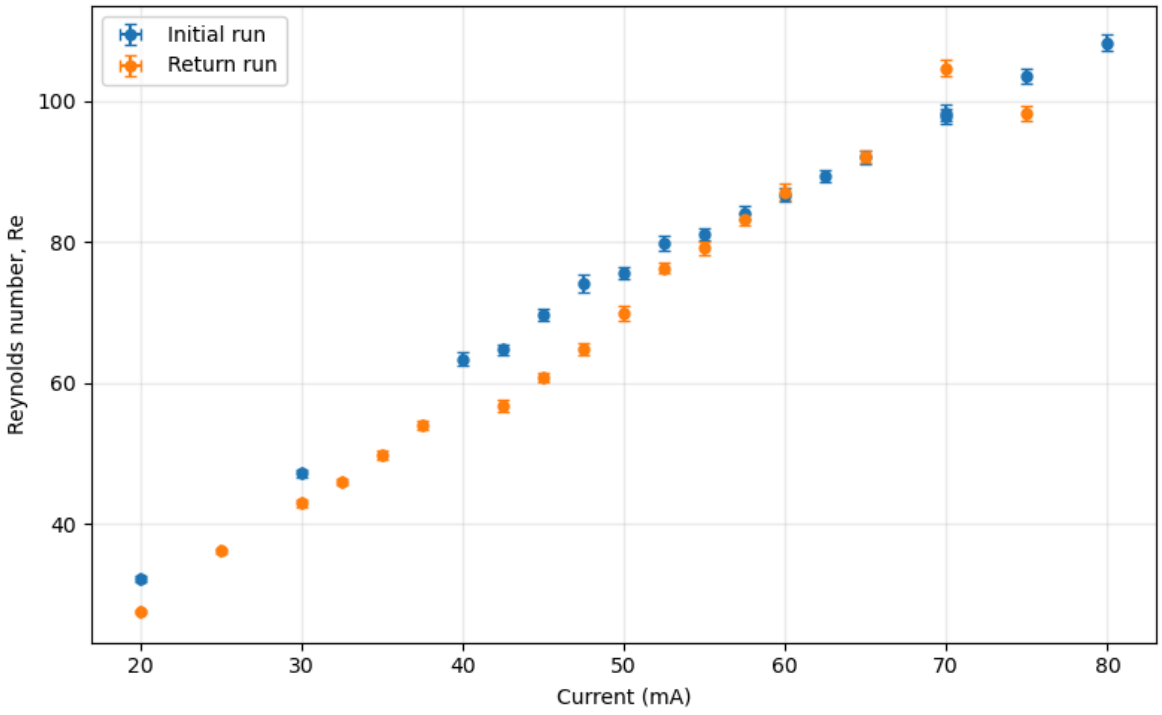
fig, ax = plt.subplots(figsize=(8, 5))

for run in runs:
    data = uncertainties[
        uncertainties["run"] == run
    ].sort_values("current_mA")
```

```
    ax.errorbar(
        data["current_mA"],
        data["Re"],
        xerr=data["current_resolution_u_mA"],
        yerr=data["Re_partial_u"],
        fmt="o",
        capsize=3,
        markersize=5,
        label=run
    )

ax.set_xlabel("Current (mA)")
ax.set_ylabel("Reynolds number, Re")
ax.grid(alpha=0.25)
ax.legend()

fig.tight_layout()
fig.savefig(
    OUTPUT_FOLDER / "reynolds_vs_current.pdf",
    dpi=300,
    bbox_inches="tight"
)
plt.show()
```




# Reynolds Number vs Cross Velocity

In [9]:
```
fig, ax = plt.subplots(figsize=(8, 5))

for run in runs:
    data = uncertainties[
        uncertainties["run"] == run
    ].sort_values("Re")

    ax.errorbar(
        data["Re"],
```

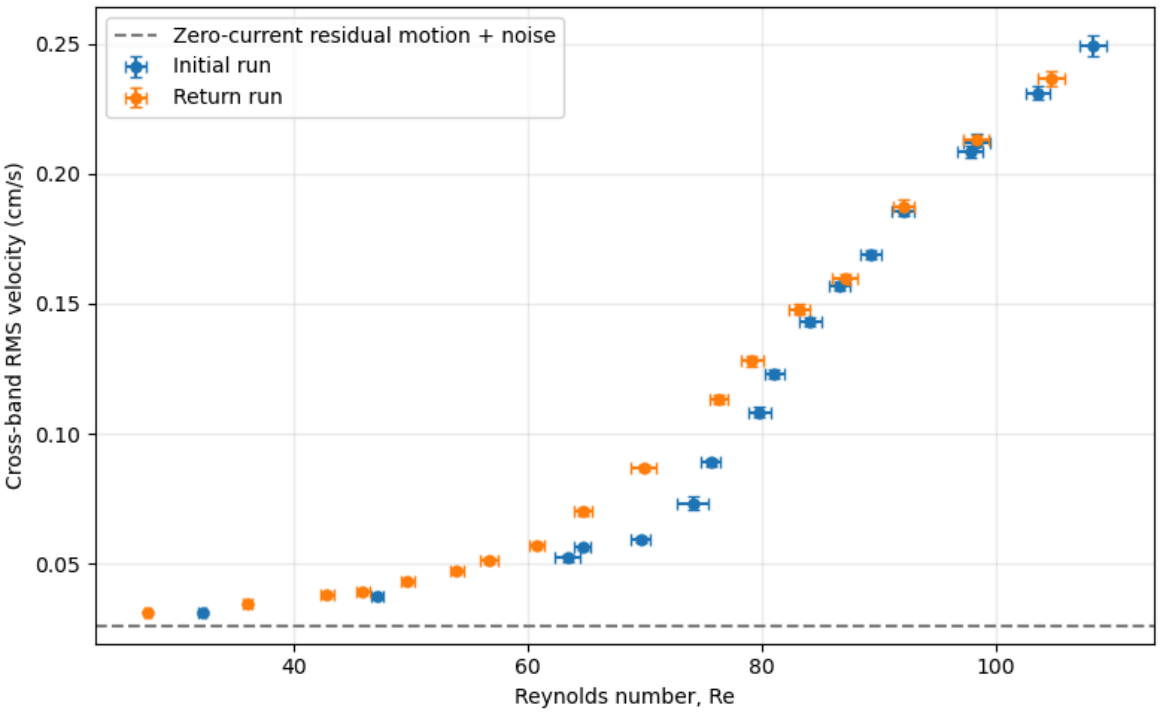
```
        data["cross_cm_s"],
        xerr=data["Re_partial_u"],
        yerr=data["cross_partial_u"],
        fmt="o",
        capsize=3,
        markersize=5,
        label=run
    )

# Zero-current reference.
zero = uncertainties[
    uncertainties["recording"] == zero_reference
]

if not zero.empty:
    baseline = zero.iloc[0]["cross_cm_s"]
    ax.axhline(
        baseline,
        color="grey",
        linestyle="--",
        label="Zero-current residual motion + noise"
    )

ax.set_xlabel("Reynolds number, Re")
ax.set_ylabel("Cross-band RMS velocity (cm/s)")
ax.grid(alpha=0.25)
ax.legend()

fig.tight_layout()
fig.savefig(
    OUTPUT_FOLDER / "cross_band_vs_reynolds.pdf",
    dpi=300,
    bbox_inches="tight"
)
plt.show()
```



# Vorticity Plots

In [18]:
```
from scipy.spatial import cKDTree
import matplotlib.pyplot as plt
import numpy as np

selected = {
    20: "20 (45).mp4",
    30: "30(47).mp4",
    40: "40(48).mp4",
    45: "45(49).mp4",
    50: "50(52).mp4",
    52.5: "52.5(54).mp4",
    60: "60m(0).mp4",
    70: "70(5).mp4",
    80: "80(6).mp4",
}

radius = 0.65   # Local fitting radius, cm.
step = 0.15     # Plotting-grid spacing, cm.
min_tracks = 6

# Use the same plotting limits for every recording.
xmin = max(
    velocities_by_file[f]["cross_cm"].min()
    for f in selected.values()
)
xmax = min(
    velocities_by_file[f]["cross_cm"].max()
    for f in selected.values()
)
ymin = max(
    velocities_by_file[f]["along_cm"].min()
    for f in selected.values()
)
ymax = min(
    velocities_by_file[f]["along_cm"].max()
    for f in selected.values()
)

x = np.arange(xmin + radius, xmax - radius, step)
y = np.arange(ymin + radius, ymax - radius, step)
X, Y = np.meshgrid(x, y)
grid = np.column_stack((X.ravel(), Y.ravel()))


def vorticity_window(data):
    xy = data[["cross_cm", "along_cm"]].to_numpy()
    uv = data[["u_cross_cm_s", "u_along_cm_s"]].to_numpy()
    tracks = data["track"].to_numpy()

    tree = cKDTree(xy)
    omega = np.full(len(grid), np.nan)

    for i, centre in enumerate(grid):
        ids = np.asarray(tree.query_ball_point(centre, radius))
```

```
        if len(ids) < 20:
            continue

        offset = (xy[ids] - centre) / radius
        track_ids, inverse = np.unique(
            tracks[ids], return_inverse=True
        )

        if len(track_ids) < min_tracks:
            continue

        # Require data on both sides of the fitting point.
        if (
            offset[:, 0].min() > -0.25
            or offset[:, 0].max() < 0.25
            or offset[:, 1].min() > -0.25
            or offset[:, 1].max() < 0.25
        ):
            continue

        weights = np.exp(
            -np.sum(offset**2, axis=1) / (2 * 0.5**2)
        )

        # Equal total weight per track within the neighbourhood.
        weights /= np.bincount(
            inverse, weights=weights
        )[inverse]

        A = np.column_stack((
            np.ones(len(ids)), offset[:, 0], offset[:, 1]
        ))

        root_weight = np.sqrt(weights)
        Aw = A * root_weight[:, None]
        Bw = uv[ids] * root_weight[:, None]

        if np.linalg.cond(Aw) > 30:
            continue

        coefficients, _, rank, _ = np.linalg.lstsq(
            Aw, Bw, rcond=None
        )

        if rank == 3:
            # omega = dv/dx - du/dy, in s^-1.
            omega[i] = (
                coefficients[1, 1] - coefficients[2, 0]
            ) / radius

    return omega.reshape(X.shape)


vorticity_maps = {}

for current, filename in selected.items():
    data = velocities_by_file[filename].replace(
        [np.inf, -np.inf], np.nan
    ).dropna(subset=[
        "window", "track", "cross_cm", "along_cm",
```

```
        "u_cross_cm_s", "u_along_cm_s"
    ])

    fields = [
        vorticity_window(window)
        for _, window in data.groupby("window")
    ]

    if not fields:
        raise ValueError(f"No usable data: {filename}")

    fields = np.stack(fields)

    # Average windows equally and retain supported points only.
    supported = np.all(np.isfinite(fields), axis=0)
    mean_field = np.full(X.shape, np.nan)
    mean_field[supported] = fields[:, supported].mean(axis=0)

    vorticity_maps[current] = mean_field
    print(f"{current} mA complete")

values = np.concatenate([
    field[np.isfinite(field)]
    for field in vorticity_maps.values()
])

limit = max(np.max(np.abs(values)), 1e-6)
levels = np.linspace(-limit, limit, 61)

fig, axes = plt.subplots(
    2, 4, figsize=(17, 9), constrained_layout=True
)
axes = axes.ravel()

for ax, (current, field) in zip(axes, vorticity_maps.items()):
    ax.set_facecolor("0.9")

    ax.contourf(
        X, Y, np.ma.masked_invalid(field),
        levels=levels,
        cmap="RdBu_r",
        corner_mask=False
    )

    ax.set(
        title=f"{current} mA",
        xlabel="Across-band position (cm)",
        ylabel="Along-band position (cm)",
        xlim=(xmin, xmax),
        ylim=(ymin, ymax)
    )
    ax.set_aspect("equal")

axes[-1].axis("off")

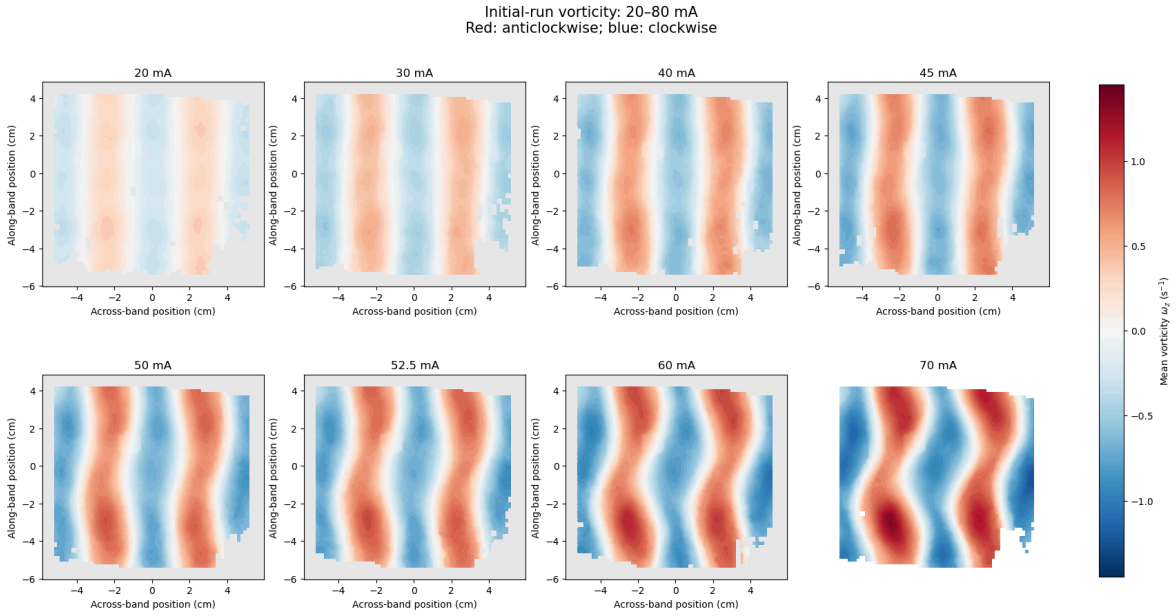
fig.colorbar(
    plt.cm.ScalarMappable(
        norm=plt.Normalize(-limit, limit),
        cmap="RdBu_r"
    ),
```

```
    ax=list(axes[:7]),
    shrink=0.85,
    label=r"Mean vorticity $\omega_z$ (s$^{-1}$)"
)

fig.suptitle(
    "Initial-run vorticity: 20–80 mA\n"
    "Red: anticlockwise; blue: clockwise",
    fontsize=15
)

fig.savefig(
    OUTPUT_FOLDER / "initial_run_vorticity_20_to_80.png",
    dpi=300,
    bbox_inches="tight"
)
plt.show()
# The fitting radius determines spatial resolution, not the grid spacing.
# Grey areas lack sufficient data; they do not represent zero vorticity.
```

```
20 mA complete
30 mA complete
40 mA complete
45 mA complete
50 mA complete
52.5 mA complete
60 mA complete
70 mA complete
80 mA complete
```




In [15]:
```
selected = {
    42.5: "47.5(50).mp4",
    45.0: "45(49).mp4",
    47.5: "42.5(51).mp4",
}

# Common plotting region for the three recordings.
xmin = max(
    velocities_by_file[f]["cross_cm"].min()
    for f in selected.values()
)
xmax = min(
```

```
    velocities_by_file[f]["cross_cm"].max()
    for f in selected.values()
)
ymin = max(
    velocities_by_file[f]["along_cm"].min()
    for f in selected.values()
)
ymax = min(
    velocities_by_file[f]["along_cm"].max()
    for f in selected.values()
)

radius = 0.65
step = 0.15
min_tracks = 6

x = np.arange(xmin + radius, xmax - radius, step)
y = np.arange(ymin + radius, ymax - radius, step)
X, Y = np.meshgrid(x, y)
grid = np.column_stack((X.ravel(), Y.ravel()))

near_onset_maps = {}

for current, filename in selected.items():
    data = velocities_by_file[filename].replace(
        [np.inf, -np.inf], np.nan
    ).dropna(subset=[
        "window", "track", "cross_cm", "along_cm",
        "u_cross_cm_s", "u_along_cm_s"
    ])

    fields = [
        vorticity_window(window)
        for _, window in data.groupby("window")
    ]

    if not fields:
        raise ValueError(f"No usable data: {filename}")

    fields = np.stack(fields)
    supported = np.all(np.isfinite(fields), axis=0)

    mean_field = np.full(X.shape, np.nan)
    mean_field[supported] = fields[:, supported].mean(axis=0)

    near_onset_maps[current] = mean_field
    print(f"{current} mA complete")

values = np.concatenate([
    field[np.isfinite(field)]
    for field in near_onset_maps.values()
])

if not len(values):
    raise ValueError("Insufficient coverage. Try radius = 0.8.")

# Shared colour scale for these three maps.
limit = max(np.max(np.abs(values)), 1e-6)
levels = np.linspace(-limit, limit, 61)
```

```
fig, axes = plt.subplots(
    1, 3, figsize=(13, 5), constrained_layout=True
)

for ax, (current, field) in zip(axes, near_onset_maps.items()):
    ax.set_facecolor("0.9")

    ax.contourf(
        X, Y,
        np.ma.masked_invalid(field),
        levels=levels,
        cmap="RdBu_r",
        corner_mask=False
    )

    ax.set(
        xlabel="Across-band position (cm)",
        ylabel="Along-band position (cm)",
        xlim=(xmin, xmax),
        ylim=(ymin, ymax)
    )
    ax.set_aspect("equal")

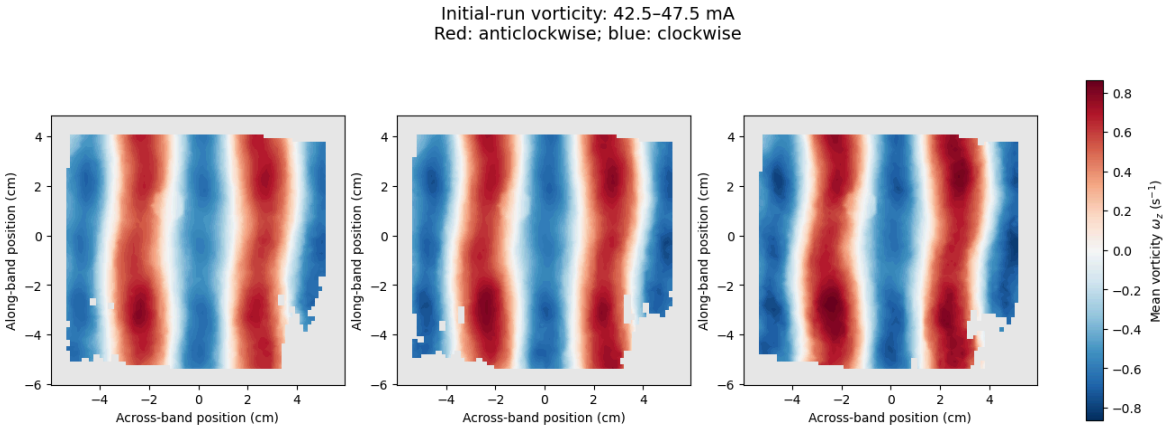
fig.colorbar(
    plt.cm.ScalarMappable(
        norm=plt.Normalize(-limit, limit),
        cmap="RdBu_r"
    ),
    ax=list(axes),
    shrink=0.85,
    label=r"Mean vorticity $\omega_z$ (s$^{-1}$)"
)

fig.suptitle(
    "Initial-run vorticity: 42.5–47.5 mA\n"
    "Red: anticlockwise; blue: clockwise",
    fontsize=14
)

fig.savefig(
    OUTPUT_FOLDER / "initial_run_vorticity_42p5_to_47p5.png",
    dpi=300,
    bbox_inches="tight"
)
plt.show()

# Grey areas indicate insufficient data support.
# This figure's colour range differs from the earlier 20–80 mA figure.
```

```
42.5 mA complete
45.0 mA complete
47.5 mA complete
```




# Breakdown Estimate Forwards Run

In [24]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import odr

data = uncertainties[
    uncertainties["run"] == "Initial run"
].sort_values("Re").copy()

x = data["Re"].to_numpy()
y = data["cross_cm_s"].to_numpy()

xerr = data["Re_partial_u"].to_numpy()
yerr = data["cross_partial_u"].to_numpy()


def predict(x, breakpoint, coefficients):
    a, b, c = coefficients
    return a + b*x + c*np.maximum(0, x - breakpoint)


def fit_breakpoint(x, y, xerr, yerr, n_candidates=1000):

    unique_x = np.unique(x)

    candidates = np.linspace(
        unique_x[2] + 1e-6,
        unique_x[-3] - 1e-6,
        n_candidates
    )

    best = None

    for breakpoint in candidates:

        def model_func(beta, x_odr):
            a, b, c = beta
            return (
                a
                + b*x_odr
                + c*np.maximum(0, x_odr - breakpoint)
            )
```

```
        model = odr.Model(model_func)

        real_data = odr.RealData(
            x,
            y,
            sx=xerr,
            sy=yerr
        )

        A = np.column_stack((
            np.ones(len(x)),
            x,
            np.maximum(0, x - breakpoint)
        ))

        beta0 = np.linalg.lstsq(
            A,
            y,
            rcond=None
        )[0]

        fit = odr.ODR(
            real_data,
            model,
            beta0=beta0
        ).run()

        coefficients = fit.beta
        error = fit.sum_square

        if best is None or error < best[0]:
            best = (
                error,
                breakpoint,
                coefficients,
                fit
            )

    return best


error, breakpoint, coefficients, odr_fit = fit_breakpoint(
    x,
    y,
    xerr,
    yerr,
    n_candidates=1000
)

fitted = predict(
    x,
    breakpoint,
    coefficients
)

residuals = y - fitted


left_mask = x <= breakpoint
right_mask = x > breakpoint
```

```
sigma_left = np.std(
    residuals[left_mask],
    ddof=1
)

sigma_right = np.std(
    residuals[right_mask],
    ddof=1
)

branch_sigma = np.where(
    left_mask,
    sigma_left,
    sigma_right
)

normalised_residuals = (
    residuals / branch_sigma
)


rng = np.random.default_rng(42)

n_mc = 500
mc_breakpoints = []

for i in range(n_mc):

    x_mc = rng.normal(
        loc=x,
        scale=xerr
    )

    y_mc = rng.normal(
        loc=y,
        scale=yerr
    )

    order = np.argsort(x_mc)

    x_mc = x_mc[order]
    y_mc = y_mc[order]
    xerr_mc = xerr[order]
    yerr_mc = yerr[order]

    try:
        _, bp_mc, _, _ = fit_breakpoint(
            x_mc,
            y_mc,
            xerr_mc,
            yerr_mc,
            n_candidates=240
        )

        mc_breakpoints.append(bp_mc)

    except Exception:
        pass
```

```
mc_breakpoints = np.array(mc_breakpoints)

if len(mc_breakpoints) == 0:
    raise RuntimeError(
        "No successful Monte Carlo breakpoint fits."
    )


bp_16, bp_median, bp_84 = np.percentile(
    mc_breakpoints,
    [16, 50, 84]
)

bp_unc_minus = breakpoint - bp_16
bp_unc_plus = bp_84 - breakpoint


xx = np.linspace(
    x.min(),
    x.max(),
    500
)

yy = predict(
    xx,
    breakpoint,
    coefficients
)

sigma_band = np.where(
    xx <= breakpoint,
    sigma_left,
    sigma_right
)

lower_band = yy - sigma_band
upper_band = yy + sigma_band


fig, (ax, residual_ax) = plt.subplots(
    2,
    1,
    figsize=(8, 7),
    sharex=True,
    gridspec_kw={
        "height_ratios": [3, 1]
    },
    constrained_layout=True
)


ax.errorbar(
    x,
    y,
    xerr=xerr,
    yerr=yerr,
    fmt="o",
    color="tab:blue",
    ecolor="tab:blue",
```

```
    markersize=5,
    capsize=3,
    label="Initial run",
    zorder=4
)


ax.fill_between(
    xx,
    lower_band,
    upper_band,
    color="black",
    alpha=0.12,
    label=r"$\pm1\sigma$ residual spread",
    zorder=1
)


ax.plot(
    xx,
    yy,
    color="black",
    linewidth=2.2,
    label=(
        f"Two-slope ODR fit: "
        f"$Re_b={breakpoint:.1f}"
        f"^{{+{bp_unc_plus:.1f}}}"
        f"_{{-{bp_unc_minus:.1f}}}$"
    ),
    zorder=3
)


ax.axvline(
    breakpoint,
    color="black",
    linestyle=":",
    linewidth=1.5,
    zorder=2
)


ax.set_ylabel(
    "Cross-band RMS velocity (cm/s)"
)

ax.grid(alpha=0.25)
ax.legend(fontsize=9)


residual_ax.scatter(
    x,
    normalised_residuals,
    color="tab:blue",
    s=25,
    zorder=3
)

residual_ax.axhline(
    0,
```

```
    color="black",
    linewidth=1
)

residual_ax.set_xlabel(
    "Reynolds number, Re"
)

residual_ax.set_ylabel(
    "Normalised\nresidual"
)

residual_ax.grid(alpha=0.25)


fig.savefig(
    OUTPUT_FOLDER / "initial_run_fit_and_residuals.pdf",
    bbox_inches="tight"
)

plt.show()


print(
    f"Fitted breakpoint: Re = {breakpoint:.2f}"
)

print(
    f"Monte Carlo 68% interval: "
    f"{bp_16:.2f} to {bp_84:.2f}"
)

print(
    f"Breakpoint result: "
    f"Re = {breakpoint:.2f} "
    f"(+{bp_unc_plus:.2f}, "
    f"-{bp_unc_minus:.2f})"
)

print(
    f"Monte Carlo median breakpoint: "
    f"{bp_median:.2f}"
)

print(
    f"Successful Monte Carlo fits: "
    f"{len(mc_breakpoints)} / {n_mc}"
)

print(
    "Pre-breakpoint residual SD: "
    f"{sigma_left:.5f} cm/s"
)

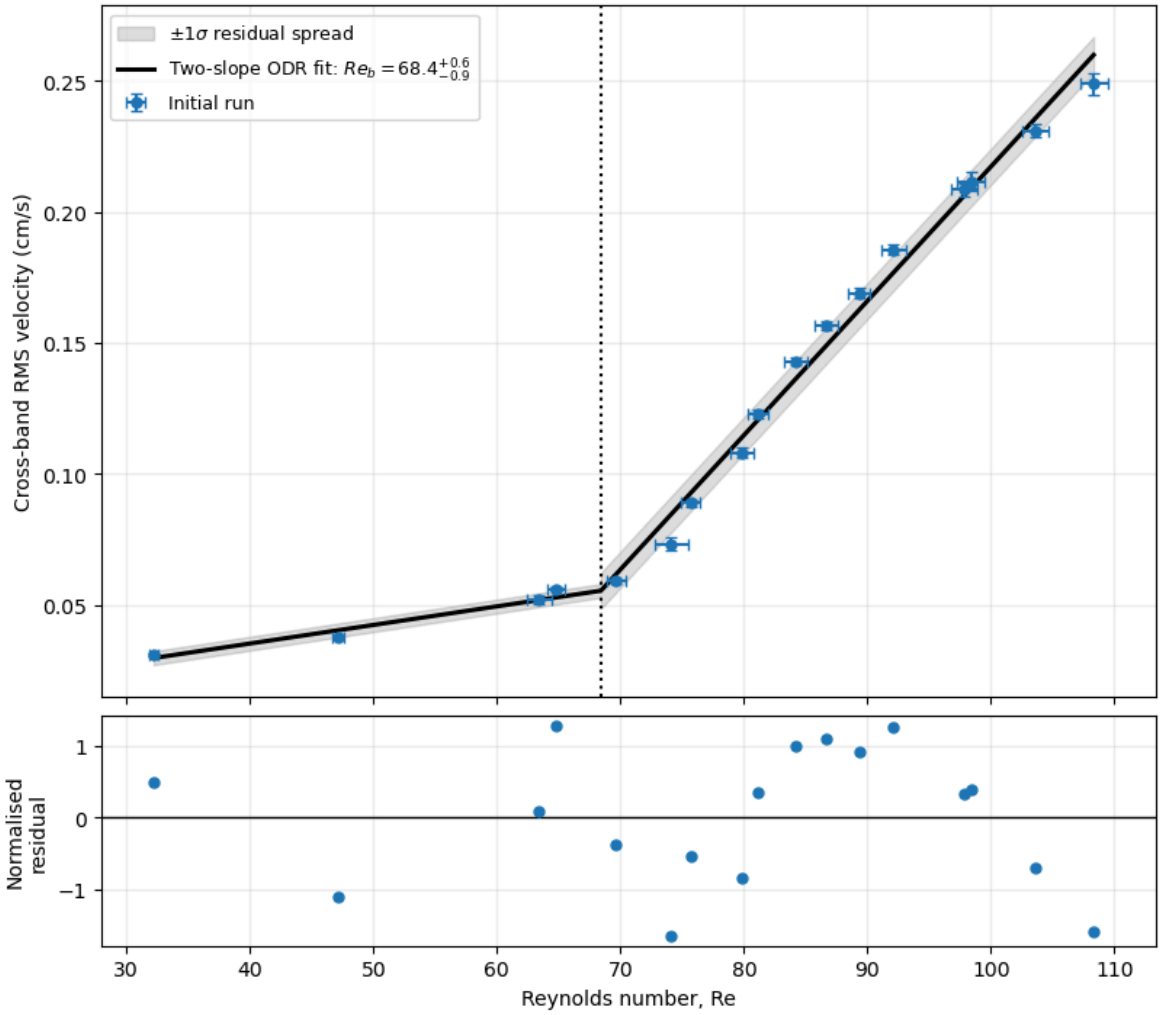
print(
    "Post-breakpoint residual SD: "
    f"{sigma_right:.5f} cm/s"
)

print(
```

```
    "Overall residual RMS: "
    f"{np.sqrt(np.mean(residuals**2)):.5f} cm/s"
)

print(
    "Normalised residual RMS: "
    f"{np.sqrt(np.mean(normalised_residuals**2)):.3f}"
)

print(
    "Two-slope ODR weighted sum: "
    f"{odr_fit.sum_square:.3f}"
)
```




```
Fitted breakpoint: Re = 68.41
Monte Carlo 68% interval: 67.54 to 69.02
Breakpoint result: Re = 68.41 (+0.61, -0.87)
Monte Carlo median breakpoint: 68.30
Successful Monte Carlo fits: 500 / 500
Pre-breakpoint residual SD: 0.00263 cm/s
Post-breakpoint residual SD: 0.00687 cm/s
Overall residual RMS: 0.00588 cm/s
Normalised residual RMS: 0.944
Two-slope ODR weighted sum: 182.102
```

# Breakdown Estimate Reverse Run

In [23]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import odr

data = uncertainties[
    uncertainties["run"] == "Return run"
].sort_values("Re").copy()

x = data["Re"].to_numpy()
y = data["cross_cm_s"].to_numpy()

xerr = data["Re_partial_u"].to_numpy()
yerr = data["cross_partial_u"].to_numpy()


def predict(x, breakpoint, coefficients):
    a, b, c = coefficients
    return a + b*x + c*np.maximum(0, x - breakpoint)


def fit_breakpoint(x, y, xerr, yerr, n_candidates=1000):

    unique_x = np.unique(x)

    candidates = np.linspace(
        unique_x[2] + 1e-6,
        unique_x[-3] - 1e-6,
        n_candidates
    )

    best = None

    for breakpoint in candidates:

        def model_func(beta, x_odr):
            a, b, c = beta
            return (
                a
                + b*x_odr
                + c*np.maximum(0, x_odr - breakpoint)
            )

        model = odr.Model(model_func)

        real_data = odr.RealData(
            x,
            y,
            sx=xerr,
            sy=yerr
        )

        A = np.column_stack((
            np.ones(len(x)),
            x,
            np.maximum(0, x - breakpoint)
        ))

        beta0 = np.linalg.lstsq(
            A,
```

```
            y,
            rcond=None
        )[0]

        fit = odr.ODR(
            real_data,
            model,
            beta0=beta0
        ).run()

        coefficients = fit.beta
        error = fit.sum_square

        if best is None or error < best[0]:
            best = (
                error,
                breakpoint,
                coefficients,
                fit
            )

    return best


error, breakpoint, coefficients, odr_fit = fit_breakpoint(
    x,
    y,
    xerr,
    yerr,
    n_candidates=1000
)

fitted = predict(
    x,
    breakpoint,
    coefficients
)

residuals = y - fitted


left_mask = x <= breakpoint
right_mask = x > breakpoint

sigma_left = np.std(
    residuals[left_mask],
    ddof=1
)

sigma_right = np.std(
    residuals[right_mask],
    ddof=1
)

branch_sigma = np.where(
    left_mask,
    sigma_left,
    sigma_right
)
```

```
normalised_residuals = (
    residuals / branch_sigma
)


rng = np.random.default_rng(42)

n_mc = 500
mc_breakpoints = []

for i in range(n_mc):

    x_mc = rng.normal(
        loc=x,
        scale=xerr
    )

    y_mc = rng.normal(
        loc=y,
        scale=yerr
    )

    order = np.argsort(x_mc)

    x_mc = x_mc[order]
    y_mc = y_mc[order]
    xerr_mc = xerr[order]
    yerr_mc = yerr[order]

    try:
        _, bp_mc, _, _ = fit_breakpoint(
            x_mc,
            y_mc,
            xerr_mc,
            yerr_mc,
            n_candidates=240
        )

        mc_breakpoints.append(bp_mc)

    except Exception:
        pass


mc_breakpoints = np.array(mc_breakpoints)

if len(mc_breakpoints) == 0:
    raise RuntimeError(
        "No successful Monte Carlo breakpoint fits."
    )


bp_16, bp_median, bp_84 = np.percentile(
    mc_breakpoints,
    [16, 50, 84]
)

bp_unc_minus = breakpoint - bp_16
bp_unc_plus = bp_84 - breakpoint
```

```
xx = np.linspace(
    x.min(),
    x.max(),
    500
)

yy = predict(
    xx,
    breakpoint,
    coefficients
)

sigma_band = np.where(
    xx <= breakpoint,
    sigma_left,
    sigma_right
)

lower_band = yy - sigma_band
upper_band = yy + sigma_band


fig, (ax, residual_ax) = plt.subplots(
    2,
    1,
    figsize=(8, 7),
    sharex=True,
    gridspec_kw={
        "height_ratios": [3, 1]
    },
    constrained_layout=True
)


ax.errorbar(
    x,
    y,
    xerr=xerr,
    yerr=yerr,
    fmt="o",
    color="tab:orange",
    ecolor="tab:orange",
    markersize=5,
    capsize=3,
    label="Return run",
    zorder=4
)


ax.fill_between(
    xx,
    lower_band,
    upper_band,
    color="black",
    alpha=0.12,
    label=r"$\pm1\sigma$ residual spread",
    zorder=1
)
```

```
ax.plot(
    xx,
    yy,
    color="black",
    linewidth=2.2,
    label=(
        f"Two-slope ODR fit: "
        f"$Re_b={breakpoint:.1f}"
        f"^{{+{bp_unc_plus:.1f}}}"
        f"_{{-{bp_unc_minus:.1f}}}$"
    ),
    zorder=3
)


ax.axvline(
    breakpoint,
    color="black",
    linestyle=":",
    linewidth=1.5,
    zorder=2
)


ax.set_ylabel(
    "Cross-band RMS velocity (cm/s)"
)

ax.grid(alpha=0.25)
ax.legend(fontsize=9)


residual_ax.scatter(
    x,
    normalised_residuals,
    color="tab:orange",
    s=25,
    zorder=3
)

residual_ax.axhline(
    0,
    color="black",
    linewidth=1
)

residual_ax.set_xlabel(
    "Reynolds number, Re"
)

residual_ax.set_ylabel(
    "Normalised\nresidual"
)

residual_ax.grid(alpha=0.25)


fig.savefig(
    OUTPUT_FOLDER / "return.pdf",
```

```
    bbox_inches="tight"
)

plt.show()


print(
    f"Fitted breakpoint: Re = {breakpoint:.2f}"
)

print(
    f"Monte Carlo 68% interval: "
    f"{bp_16:.2f} to {bp_84:.2f}"
)

print(
    f"Breakpoint result: "
    f"Re = {breakpoint:.2f} "
    f"(+{bp_unc_plus:.2f}, "
    f"-{bp_unc_minus:.2f})"
)

print(
    f"Monte Carlo median breakpoint: "
    f"{bp_median:.2f}"
)

print(
    f"Successful Monte Carlo fits: "
    f"{len(mc_breakpoints)} / {n_mc}"
)

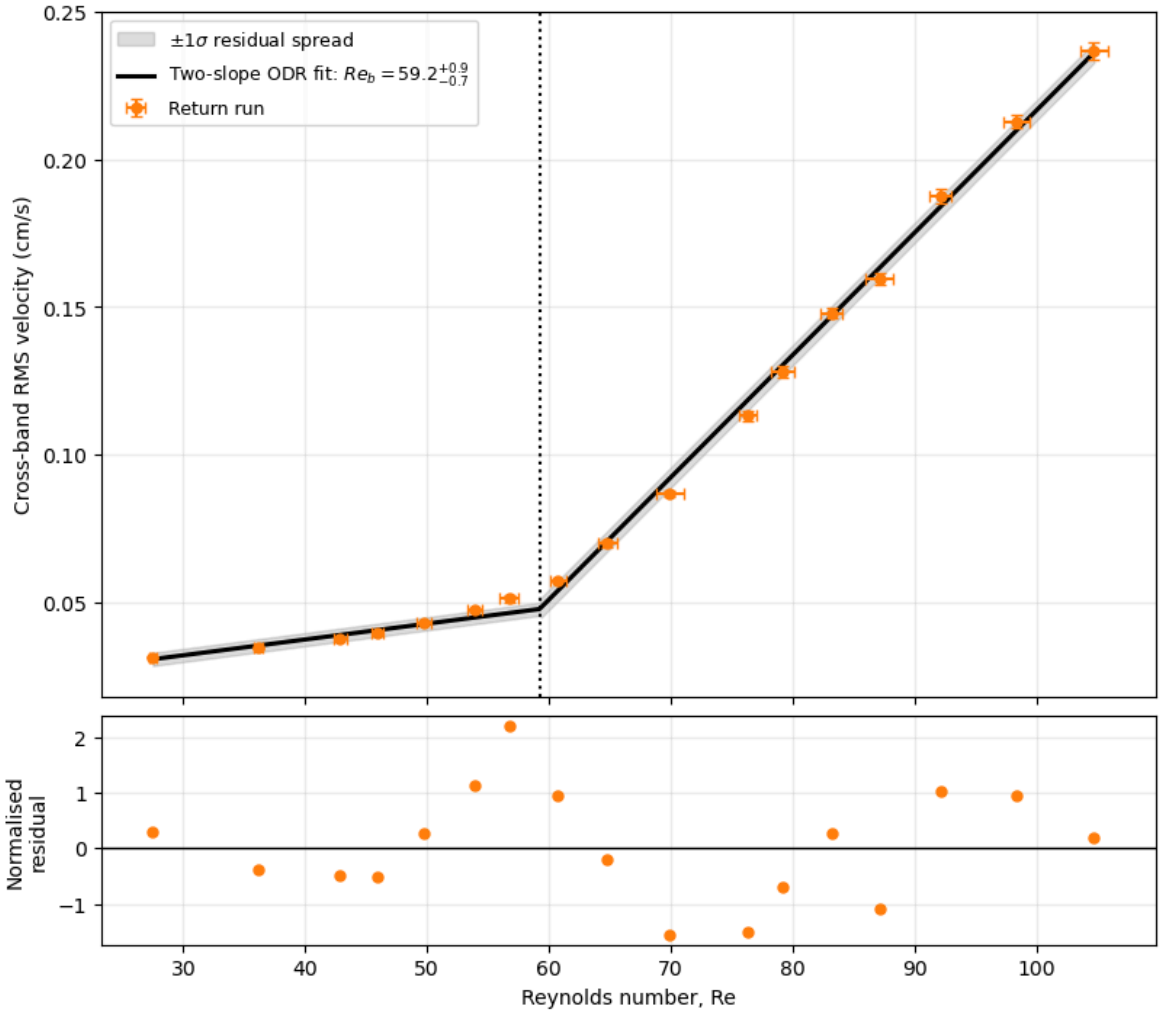
print(
    "Pre-breakpoint residual SD: "
    f"{sigma_left:.5f} cm/s"
)

print(
    "Post-breakpoint residual SD: "
    f"{sigma_right:.5f} cm/s"
)

print(
    "Overall residual RMS: "
    f"{np.sqrt(np.mean(residuals**2)):.5f} cm/s"
)

print(
    "Normalised residual RMS: "
    f"{np.sqrt(np.mean(normalised_residuals**2)):.3f}"
)

print(
    "Two-slope ODR weighted sum: "
    f"{odr_fit.sum_square:.3f}"
)
```




```
Fitted breakpoint: Re = 59.24
Monte Carlo 68% interval: 58.54 to 60.14
Breakpoint result: Re = 59.24 (+0.90, -0.71)
Monte Carlo median breakpoint: 59.22
Successful Monte Carlo fits: 500 / 500
Pre-breakpoint residual SD: 0.00221 cm/s
Post-breakpoint residual SD: 0.00342 cm/s
Overall residual RMS: 0.00289 cm/s
Normalised residual RMS: 0.975
Two-slope ODR weighted sum: 121.954
```

In [19]:

```python
currents = [30, 40, 45, 52.5]

re_values = {
    30: 47.1,
    40: 63.4,
    45: 69.7,
    52.5: 79.8
}

# Shared colour scale for these four maps.
values = np.concatenate([
    vorticity_maps[i][np.isfinite(vorticity_maps[i])]
    for i in currents
])

colour_limit = max(np.max(np.abs(values)), 1e-6)
colour_levels = np.linspace(-colour_limit, colour_limit, 61)
```

```
fig, axes = plt.subplots(
    2, 2, figsize=(9, 8), constrained_layout=True
)

for ax, current, letter in zip(axes.ravel(), currents, "abcd"):
    field = vorticity_maps[current]

    ax.set_facecolor("0.9")

    ax.contourf(
        X, Y,
        np.ma.masked_invalid(field),
        levels=colour_levels,
        cmap="RdBu_r",
        corner_mask=False
    )

    ax.set(
        xlabel="Across-band position (cm)",
        ylabel="Along-band position (cm)",
        xlim=(xmin, xmax),
        ylim=(ymin, ymax)
    )

    ax.set_aspect("equal")

    # Panel label in top-left
    ax.text(
        0.04, 0.96,
        f"({letter})",
        transform=ax.transAxes,
        ha="left",
        va="top",
        fontsize=14,
        fontweight="bold"
    )

fig.colorbar(
    plt.cm.ScalarMappable(
        norm=plt.Normalize(-colour_limit, colour_limit),
        cmap="RdBu_r"
    ),
    ax=list(axes.ravel()),
    shrink=0.85,
    label=r"Mean vorticity $\omega_z$ (s$^{-1}$)"
)

save_path = OUTPUT_FOLDER / "vorticityrey.pdf"

fig.savefig(
    save_path,
    format="pdf",
    bbox_inches="tight"
)

plt.close(fig)

print(f"Saved: {save_path}")
```

```
Saved: C:\Users\lewis\OneDrive\Documents\Lab reports\New folder\particle_tracking
_results\rotated_results\vorticityrey.pdf
```

In [ ]: